

RemSolution — Complete Roadmap v2 (Consolidated)

July 13, 2026 · Supersedes the July 3 roadmap and the July 13 amendments document — this is now the single source of truth.

Multi-tenant SaaS for rental agencies + public Airbnb-style marketplace. .NET 10 / Angular / SQL Server, Clean Architecture, single database shared schema with AgencyId global query filters.

Status assumption: Phases 0, 0.5 and 1 are complete as originally specified. Amendments defined after those phases shipped (Branch, MaxUsers, permission system, ...) appear below as **Backfill** tasks — still to do.

Legend: ✓ **Done** · ■ **Backfill** (amendment to a completed phase) · ■ **To do**

Business model (locked)

- **Platform admin (one, seeded):** countries, brands, models; creates agencies + their admin; assigns subscription plans (MaxCars, MaxUsers, MaxClients); enables feature modules per agency (ExtraServices, Expenses, Statistics); read-only cross-tenant visibility via a dedicated audited path.
- **Agency admin:** cars assigned to Branches (pickup locations with geography); agency users + per-user CRUD permissions; contract/invoice templates; ExtraServiceType and ExpenseType (threshold by months OR mileage) if features enabled; statistics if enabled. Holds all permissions implicitly.
- **Agency user:** clients, rentals, reservations, payments, expenses, extra services — each action gated by agency feature flag AND user permission, enforced at the API.
- **Marketplace client:** picks country/place + dates → available cars from nearby branches → books (forced account: email + phone; uploads passport + driving licence) → reservation PendingConfirmation (holds the car, with expiry) → agency confirms → payment → pickup/conversion → afterwards: in-app messaging, rating (car/agency, anchored to completed renting), reporting.
- **Mobile (later):** single role-based app (decision pending); agency scans client documents (OCR) → auto-fills the contract. Only requirement today: store document originals at full resolution.

Phase 0 — Hygiene ✓ Done (2–3 days)

#	Task	Status
0.1	Retarget to .NET 10 LTS, bump packages, rebuild, re-run tests	✓ Done
0.2	Squash EF migration (drop Todo tables, remove PendingModelChangesWarning suppression)	✓ Done
0.3	Delete Domain.User; ApplicationUser as single identity source; domain references users by string UserId	✓ Done
0.4	Externalize connection string (user secrets / Key Vault)	✓ Done
0.5	Delete template-leftover domain events + PriorityLevel enum	✓ Done
0.6	Drop Bootstrap, keep Angular Material only	✓ Done
0.7	Update commands for Brand, ModelCar, Country; seed countries	✓ Done

Phase 0.5 — Multi-tenancy, subscriptions & logging ✓ Done + ■ Backfill (4–5 days done, +2–2.5 days backfill)

#	Task	Status
T.1	Agency root entity, NetTopologySuite enabled, platform-admin CRUD	✓ Done
T.2	AgencyId on every tenant table; tenant-scoped indexes lead with AgencyId	✓ Done
T.3	ITenantProvider from JWT claim; global query filters; SaveChanges interceptor stamps AgencyId; IgnoreQueryFilters() restricted to MarketplaceSearch + audited platform-admin path	✓ Done
T.4	SubscriptionPlan (MaxCars, MaxClients) + AgencySubscription; limits enforced under sp_getapplock; expired → block writes	✓ Done

#	Task	Status
T.5	Roles: PlatformAdministrator / AgencyAdministrator / AgencyStaff; AgencyId claim; seeded platform admin	✓ Done
T.6	Serilog structured logging (AgencyId/UserId/CorrelationId enrichers) + AuditLog table via [Auditable] pipeline behaviour	✓ Done
B.1	Branch entity — Branch(Id, AgencyId, Name, CountryId, Location geography), CRUD for agency admin, spatial index on Location, IX_Branch(AgencyId, ...). The geographic anchor moves from Agency to Branch (Agency keeps HQ address only, no spatial role).	■ Backfill 0.5 d
B.2	Permission system replaces fixed AgencyStaff role — AgencyFeature(AgencyId, Feature, Enabled) + UserPermission(UserId, Permission) (or Identity claims: Client.Create, Renting.Delete, ...), loaded into JWT at login, enforced by policies + [RequiresFeature(...)] pipeline behaviour (same pattern as [Auditable]). Feature off ⇒ 403 at API and module hidden in Angular. AgencyAdministrator = all permissions implicitly. Admin UI can come later; tables + plumbing now.	■ Backfill 1 d
B.3	SubscriptionPlan.MaxUsers + enforcement in CreateAgencyUser under the same sp_getapplock discipline	■ Backfill 0.3 d
B.4	Add platform-admin cross-tenant reads explicitly to the [Auditable] list	■ Backfill 0.1 d

Phase 1 — Client vertical slice ✓ Done + ■ Backfill (1 week done, +0.5 day backfill)

#	Task	Status
1.1	Client CRUD commands/queries, vertical-slice layout, Mapster projections, nullable MarketplaceUserId reserved	✓ Done
1.2	FluentValidation: CIN / Passport / Driving Licence blocks, dates, Country FKs	✓ Done
1.3	IFileStorage abstraction; paths in SQL + files in blob/wwwroot	✓ Done
1.4	DeleteBehavior.Restrict on multiple Country FKs	✓ Done
1.5	Angular Client CRUD (Material table, forms, image upload), NSwag regenerated	✓ Done
B.5	Verify IFileStorage stores originals at full resolution (mobile OCR later); add document type tagging (Passport, DrivingLicence, CIN) on stored files	■ Backfill 0.3 d
B.6	Client.IsFlagged + Client.Notes (per-agency bad-client flag; cross-agency blacklist deliberately excluded — legal minefield)	■ Backfill 0.2 d

Pre-Phase-2 gate ■ To do (blocking decisions + schema, ~1.5–2 days)

Everything here touches the Car schema or the Money model — must land before the Renting aggregate exists.

#	Task	Detail	Effort
P.1	Car.BranchId FK	Every car belongs to a pickup Branch (requires B.1). Availability and marketplace search anchor on Branch, never raw coordinates on Car.	0.2 d
P.2	Car.Status	Active / Maintenance / Inactive — cars drop out of availability without deleting history. Filter in every availability query.	0.2 d
P.3	Car.DailyRate	Flat daily rate v1 (listed price the marketplace displays before booking). Seasonal/duration pricing explicitly deferred. Feeds search DTO (6.2) and contract generation (7.4).	0.3 d
P.4	Money value object with currency code	Amount + currency + rounding rules. v1 rule: one currency per agency (column on Agency), no conversion. Retrofitting currency onto bare decimals later = the AgencyId mistake again.	0.5 d
P.5	UTC decision	All dates stored UTC (or DateTimeOffset), converted at the edge. Branches span countries/timezones; the overlap predicate must compare like with like. Decide and document before the first StartDate is written.	0.1 d

#	Task	Detail	Effort
P.6	Car photos schema	CarImage(CarId, Path, SortOrder, IsPrimary) on IFileStorage. Upload UI lands with the car form (P.7); marketplace gallery in Phase 6. A marketplace without car pictures doesn't sell.	0.3 d
P.7	Car form update	Angular: branch picker, status, daily rate, photo upload on the existing Car screens.	0.5 d

Phase 2 — Renting core ■ To do (2–2.5 weeks)

#	Task	Detail	Effort
2.1	Renting as rich aggregate	State machine (NotYet/InProgress/Done) in the domain: private setters, factory Renting.Create(...) enforcing invariants (end > start, price ≥ 0), Start() / Complete(endMileage) guarding transitions, RentingCompletedEvent → RentingHistory snapshot. Handlers shrink to load → call → save. Uses the Money VO from P.4.	1.5 d
2.2	CreateRenting + race-safe availability	sp_getapplock keyed on CarId inside one transaction around overlap check + insert. Overlap: newStart < existingEnd AND newEnd > existingStart; decide same-day turnaround. Predicate covers Renting AND Reservation in non-terminal states from day one (marketplace books via Reservation — do not bolt this on in Phase 3). Exclude Car.Status ≠ Active.	3 d
2.3	Return / close flow	CompleteRentingCommand: end mileage ≥ start, transition to Done, final price incl. extra services, RentingHistory snapshot.	2 d
2.4	ExtraServices on renting	Attach/detach lines; ExtraServicesType CRUD (agency admin, behind ExtraServices feature flag); included in price.	1 d
2.5	Queries + availability indexes	GetRentingById, paginated filtered list, car-availability query. Indexes: IX_Renting(CarId, StartDate, EndDate) INCLUDE (State) + mirror on Reservation — keeps NOT EXISTS anti-joins fast into millions of rows.	1.5 d
2.6	Tests — DDD pyramid, as you build	Domain.UnitTests: Create rejects invalid dates/price; Complete() from NotYet throws; Complete() validates mileage + raises event; Money arithmetic/rounding/currency mismatch throws. Application.UnitTests: validators; handlers orchestrate only. FunctionalTests (Testcontainers + Respawn): overlap incl. boundary dates; two parallel CreateRenting → exactly one succeeds; RentingHistory written; subscription limit blocks N+1th car; feature flag off → 403; user without Renting.Create → 403, with it → 200.	2.5 d
2.7	Vehicle condition at pickup/return	Free-text condition notes + optional photos on Start() and Complete() (rides on IFileStorage). Feeds future dispute handling (7.3).	0.5 d
2.8	Angular Renting UI	List + detail, create wizard (car → availability → client(s) → dates/price → services), return dialog with condition capture. NSwag regen.	2 d

Phase 3 — Reservation + Payment ■ To do (1.5–2 weeks)

#	Task	Detail	Effort
3.1	Reservation as rich aggregate (state machine)	PendingConfirmation → Confirmed → Paid → Converted, plus Rejected / Expired / Cancelled — built like Renting 2.1 (factory, guarded transitions, domain events). ExpiresAt on PendingConfirmation. Participates in the shared overlap check (already wired in 2.2). Documents (passport/licence) attach to the Reservation — the marketplace user has no Client row yet at booking time.	2 d
3.2	Expiry background job	Auto-expire unconfirmed reservations after N hours (per-agency setting, default 24h). First legitimate scheduler (Hangfire or hosted service) — narrowly scoped. The PII purge job (7.5) will reuse it.	0.5 d
3.3	Convert reservation → renting	Allowed only from Confirmed/Paid; same per-car lock; carries dates/price/advance payment; creates the Client row (fills MarketplaceUserId) and moves documents onto it; links Reservation ↔ Renting.	1 d

#	Task	Detail	Effort
3.4	Payment recording	Payments against client / renting / reservation. Money VO, EF precision explicit. Invariant: PayedPrice vs Price — decide and enforce in validator. Client balance query. Refund flag on Payment. Payment provider integration out of v1; the Paid state exists regardless.	1.5 d
3.5	Deposit & cancellation policy	DepositAmount on Renting/Reservation; per-agency cancellation window (free until X hours before pickup) checked in CancelReservation. Business rule must be decided before the validators are written.	0.5 d
3.6	Angular UI	Reservation calendar/list with states + confirm/reject/convert actions, payment entry, client account view (payments vs due).	1.5 d

Phase 4 — Expense & maintenance notifications ■ To do (1 week) — cuttable

#	Task	Detail	Effort
4.1	Expense + ExpenseType CRUD	Per-car expenses (amount, date, mileage). ExpenseType: ThresholdKind (Months / Mileage) + value — exclusive choice, domain invariant, not two nullable columns . Whole module behind the Expenses feature flag.	1.5 d
4.2	'Due' dashboard query	Cars whose last expense of a type exceeds its threshold (query branches on ThresholdKind). No scheduler/push — dashboard query only.	1.5 d
4.3	Angular UI	Expense entry per car, overdue-maintenance dashboard widget.	1 d

Phase 5 — Hardening ■ To do (~1 week) — keep 5.3 even if cut

#	Task	Detail	Effort
5.1	Backfill tests by DDD layer	Domain: Reservation state machine incl. expiry, Payment/Money edges, Expense threshold logic. Application: every validator, [Auditable] and [RequiresFeature] behaviours. Functional: conversion carries advance payment; marketplace booking under concurrency. Placement rule: if a rule needs the DB to test, it lives in the wrong layer.	2 d
5.2	One E2E acceptance path	SpecFlow + Playwright, domain language: create client → rent → return → pay. One reliable E2E, not twenty flaky ones.	1.5 d
5.3	Authorization pass	Policies from permission claims + feature-flag behaviour across all endpoints; functional tests: tenant isolation (agency A requesting agency B's renting → 404), permission denial, feature-flag denial.	1 d
5.4	Seed + demo dataset	Countries, brands/models, branches, cars with photos, clients — plus the Phase 6 load-test dataset generator (reused by 6.6).	0.5 d

Phase 6 — Public marketplace ■ To do (2.5–3.5 weeks)

#	Task	Detail	Effort
6.1	Marketplace user self-registration	Public Identity flow, separate from agency staff; email + phone required; email verification. One global MarketplaceUser; Client row per agency created at conversion (3.3). No AgencyId claim.	2 d
6.2	Cross-tenant availability search	The one carefully designed query — read-only, anonymous, the ONLY IgnoreQueryFilters() besides the audited admin path. Shape: Branch filter by CountryId + Location.STDistance(@point) < @radius (spatial index), join Car (Status = Active), NOT EXISTS anti-joins on Renting + non-terminal Reservation with the overlap predicate, order by distance. Lean DTO: car, daily rate, primary photo, agency, branch, distance, rating average. No Includes.	3 d
6.3	Marketplace booking flow	Public booking → PendingConfirmation Reservation under the same per-car sp_getapplock (one lock discipline for both entry points) + document upload + expiry. Agency sees it in the normal Reservation list, confirms/rejects, converts via 3.3. v1 payment recorded manually / on pickup.	3 d

#	Task	Detail	Effort
6.4	Notifications for the confirmation loop	The flow is dead without these. IEmailSender abstraction (SMTP dev / provider later) + in-app notification list. Events: reservation created (→ agency), confirmed/rejected/expired (→ client), payment recorded.	1 d
6.5	Public endpoint hardening	Rate limiting (AddRateLimiter) on search + registration; CAPTCHA or verified email on signup; upload constraints (size, content-type sniffing, extension whitelist) on documents/photos; documents served only through an authorized endpoint — never public blob URLs.	1 d
6.6	Public Angular frontend	Country/place picker → map/list of nearby available cars for dates (with photos, rates, ratings) → detail gallery → booking with account creation + document upload → client account area (my reservations + statuses). Separate routing area; anonymous until booking. NSwag regen.	4.5 d
6.7	Search-path load sanity check	Seeded dataset at target scale (2,000 agencies / 100k cars / 1M+ renting rows); read the plan: spatial seek on Branch, index seeks (not scans) on the anti-joins. Caching / denormalized read model only if anonymous traffic later demands it — additive under CQRS, not built preemptively.	1 d

Phase 7 — Post-rental, communication & documents ■ To do (1.5–2 weeks)

#	Task	Detail	Effort
7.1	In-app messaging	Conversation(ReservationId/RentingId, AgencyId, MarketplaceUserId) + Message. Available only after a confirmed reservation. Polling first; SignalR only if needed.	2 d
7.2	Ratings	Review(RentingId unique, Target: Car / Agency, Stars, Comment). Invariant: only the marketplace user of a completed renting, once. Averages surfaced in 6.2 search results.	1 d
7.3	Reports (signalement)	Report(ReporterId, Target, Reason, Status); platform admin triage screen — first cross-tenant admin write beyond read-only, audited.	1 d
7.4	Contract & invoice generation	Agency-uploaded docx template + placeholder mapping, filled on RentingCompleted (pricing/tax now final). OpenXML or QuestPDF — decide then. Templates managed by agency admin.	2.5 d
7.5	PII retention & erasure (legal, not optional)	Passports/licences of real clients: retention policy (purge documents N months after last renting — job on the 3.2 scheduler), right-to-erasure path (anonymize Client, purge files, keep financial rows). Policy decided now, implemented here.	1 d
7.6	Statistics module	Agency dashboard (revenue, utilization, top cars/clients) behind the Statistics feature flag; platform-admin cross-tenant aggregates. Query-side only.	2 d

Explicitly deferred (not in this roadmap)

- **Online payment provider integration** — the Reservation Paid state exists; the provider is a later increment.
- **Seasonal / duration-based pricing** — v1 is flat daily rate (P.3).
- **Mobile apps + OCR contract auto-fill** — single role-based app decision pending; only prerequisite (full-resolution document storage, B.5) is handled.
- **GPS tracking** — schema impact acknowledged, deliberately out.
- **Cross-agency client blacklist** — legal minefield; per-agency flag only (B.6).
- **Push notifications / background maintenance scheduler** — dashboard query (4.2) suffices.
- **Search caching / denormalized read model** — only if 6.7 proves the need.

Timeline summary

Phase	Scope	Duration	Status	Cuttable?
0	Hygiene	2–3 d	✓ Done	—
0.5	Tenancy, subscriptions, logging	4–5 d	✓ Done	—
—	Backfill B.1–B.6	2.5–3 d	■ Backfill	No — B.1/B.2 block Phase 2
1	Client slice	1 wk	✓ Done	—
—	Pre-Phase-2 gate P.1–P.7	1.5–2 d	■ To do	No — Car schema + Money
2	Renting core	2–2.5 wk	■ To do	No
3	Reservation + Payment (state machine)	1.5–2 wk	■ To do	No — marketplace books via Reservation
4	Expense & notifications	1 wk	■ To do	Yes
5	Hardening	1 wk	■ To do	Partially — keep 5.3
6	Public marketplace	2.5–3.5 wk	■ To do	Deferable, not cuttable — the product goal
7	Post-rental, communication, documents	1.5–2 wk	■ To do	7.4/7.5 no; 7.1–7.3/7.6 deferable

Remaining work: ~10–12 weeks. (Original total 10–11 weeks; ~2 weeks delivered, new scope adds ~2–3 weeks.)

Immediate next steps, in order: B.1 (Branch) → B.2 (permissions plumbing) → B.3/B.4 → P.1–P.7 → start 2.1.

Definition of done: a marketplace client picks a country, sees available cars with photos and rates from nearby branches for their dates, books, gets confirmed and pays; the agency completes rent → return → pay with an auto-generated contract — with functional tests covering overlap logic, state transitions, subscription limits, permissions/feature flags, and tenant isolation.